

Catching Conflicts Before They Go Live: Solver-Verified Compilation of Natural-Language Smart-Home Policies

Tarun Raja
Independent Researcher
rajatarun12@gmail.com

Abstract—Smart-home platforms increasingly let people write automation policies in plain English, which a large language model (LLM) compiles into a rule language. Validation then happens one rule at a time: the output is well-formed, typed, and above a confidence threshold. Per-rule validation cannot see a rule that contradicts itself, a *modify* that a *block* always overrides, two modifiers that disagree about the same moment, or a safety property that no rule actually guarantees. We formalize the rule language of DeviceWeave and show that, because every condition constrains one context field, the set of contexts a rule matches is a finite union of axis-aligned boxes. Satisfiability, shadowing, conflict, redundancy and invariant coverage are then decidable *exactly* by interval arithmetic, with a concrete witness context for every finding (Theorems 1 and 2) and a polynomial bound for fixed numbers of fields and of atoms per rule (Proposition 1). On 1,000 random rule sets, the checker’s findings match an independent Z3 encoding exactly, and all 1,353 conflict and invariant witnesses reproduce their finding when replayed through the production evaluator. The checker runs $14\text{--}39\times$ faster than Z3, and it now gates policy authoring in our DeviceWeave deployment. We also release a 380-rule benchmark that scores compiled rules by semantic equivalence rather than by string match. Compiled faithfully, all 20 of its self-contradictory rules pass the per-rule validator; only the set-level analysis rejects them.

Index Terms—smart home, policy verification, LLM compilation, satisfiability modulo theories, interval analysis

I. INTRODUCTION

In DeviceWeave [1], a person writes “don’t run the heater when nobody is home”. An LLM compiles it to a policy in a small domain-specific language (DSL), and a validator checks the result’s structure before it is stored and enforced. The validator answers one question: is this rule well-formed? It does not answer whether the *set* of rules means what their authors intended. Four failure modes are invisible to it:

- a rule whose conditions cannot all hold (“above 80 and below 60 degrees”), which can never fire;
- a *modify* (e.g., dim the lights) inside a region that a *block* covers, so it never applies;
- two modifiers setting different parameters in overlapping regions, where the newest silently wins;
- a stated safety property (“the heater is always blocked when nobody is home”) with a gap that no rule covers.

Each one is a valid rule that goes live and silently does something other than what was asked. This paper contributes:

- 1) formal semantics for the DSL, mirroring the production evaluator, and a proof that these failure modes, together with redundancy, are decidable exactly by interval arithmetic, with witnesses and a complexity bound (Section III);
- 2) an implementation that gates policy authoring. Unsatisfiable rules and invariant counterexamples are refused; conflicts are returned as warnings with a witness (Section IV);
- 3) cross-validation against an independent satisfiability modulo theories (SMT) encoding in Z3 [2], and replay of every witness through the real evaluator (Section V);
- 4) a 380-rule benchmark for LLM policy compilation, scored by semantic equivalence, and a measurement of the per-rule validator’s blind spot on it (Section VI).

II. RELATED WORK

Conflict detection among home-automation rules has been studied for trigger-action programming. Examples include analysis of IFTTT-style (“If This Then That”) rules and model checking of SmartThings apps [3], [4], which typically compile to a model checker or SMT solver. Firewall-policy analysis addresses the same shadowing and redundancy questions over header fields [5]. Our contribution is observing that DeviceWeave’s DSL is in the fragment where each constraint is *unary*, which makes these questions solvable exactly without a solver in the serving path. We still cross-check against one. Where LLM compilation to a rule language is evaluated by comparing output to a reference, a string or syntactic comparison counts $hour \geq 22$ and $hour > 21$ as different; we score by semantic equivalence, so they count as equal.

III. SEMANTICS AND EXACT ANALYSIS

A. The DSL

The context has fields F with domains: temperature $[-80, 150]^\circ\text{F}$, humidity and cloud cover $[0, 100]$, hour $\{0, \dots, 23\}$, and two booleans (is_home , $is_overcast$). Let $d = |F| = 6$ and $\mathcal{D} = \prod_{f \in F} D_f$. An *atom* is (f, op, v) with $op \in \{<, \leq, >, \geq, =, \neq\}$ for numeric f , and $\{=, \neq\}$ for boolean f . A *rule* r has a device type, a conjunction of atoms A_r , an action type $\in \{block, modify, allow\}$ and parameters. Its *match set* is $\mathcal{M}(r) = \{x \in \mathcal{D} : x \models a \ \forall a \in A_r\}$.

Definition 1 (Evaluator semantics). For device δ , context x , and rules ordered newest first, the verdict for switching on is: *block* if some block rule of δ has $x \in \mathcal{M}(r)$; otherwise *modify* with the parameters of the newest matching modifier, if one exists; otherwise *allow*. Allow rules never override a block.

This is exactly the behavior of the production evaluator when every field is present in x . A missing field makes an atom false, which can only prevent a rule from matching.

B. Boxes

A *box* is $B = \prod_f I_f$, where each $I_f \subseteq D_f$ is an interval (open, closed or half-open; integer-valued for hour) or a subset of $\{\top, \perp\}$ for booleans.

Lemma 1 (Match sets are finite unions of boxes). *For every rule r with k atoms, $\mathcal{M}(r)$ is a union of at most 2^k pairwise disjoint boxes.*

Proof. Each numeric atom's satisfying set in D_f is one interval, or two disjoint intervals for $\neq$. Each boolean atom's is one subset of $\{\top, \perp\}$. An atom on f leaves the other fields unconstrained, so its satisfying set in $\mathcal{D}$ is a union of at most two disjoint boxes. Intersecting a union of disjoint boxes with another such union gives the union of pairwise intersections. These are again disjoint, and the intersection of two boxes is a box, computed fieldwise because interval intersection is an interval. The count at most doubles per atom. $\square$

Lemma 2 (Exact box difference). *For boxes A, B , the procedure that splits A along each field in turn, keeping the parts of A_f outside B_f and narrowing the rest to $A_f \cap B_f$, returns at most $2d$ pairwise disjoint boxes whose union is exactly $A \setminus B$.*

Proof. Process fields $f_1, \dots, f_d$ in order and maintain a remainder box $R_0 = A$. At step j , $R_{j-1} \setminus B_{f_j}$ splits into at most two intervals. For booleans it is a single set difference; for intervals, the part below and the part above B_{f_j} , with endpoints flipped between open and closed so no point is lost or duplicated. These give at most two output pieces, R_{j-1} with its f_j -coordinate replaced, and then R_j is R_{j-1} with f_j narrowed to $A_{f_j} \cap B_{f_j}$. A point $x \in A$ lies outside B iff it fails some coordinate. Let j be the first such coordinate: then x belongs to exactly the piece emitted at step j , since earlier steps narrowed to coordinates x satisfies. A point in $A \cap B$ reaches R_d and is never emitted. So the pieces are disjoint and cover exactly $A \setminus B$. $\square$

C. Findings as set predicates

For device δ , let $\mathcal{B}$ be the union of its block rules' match sets, and for a modifier r let $\mathcal{N}(r)$ be the union of its newer modifiers' match sets.

- *unsatisfiable*(r): $\mathcal{M}(r) = \emptyset$.
- *shadowed*(r), for r a modifier: $\mathcal{M}(r) \subseteq \mathcal{B} \cup \mathcal{N}(r)$.
- *conflict_block_allow*(a, b): $\mathcal{M}(a) \cap \mathcal{M}(b) \neq \emptyset$ for an allow a and block b .
- *conflict_modify*(m', m): $(\mathcal{M}(m') \cap \mathcal{M}(m)) \setminus \mathcal{B} \neq \emptyset$ for m' newer than m with different parameters.

- *redundant*(r): $\mathcal{M}(r) \subseteq \bigcup \{\mathcal{M}(r') : r' \neq r \text{ has the same action (and parameters)}\}$.
- *invariant_violated*(I) for $I = (\delta, A_I)$: $\mathcal{M}(I) \setminus \mathcal{B} \neq \emptyset$.

Theorem 1 (Exactness). *The checker reports each finding if and only if its set predicate holds.*

Proof. By Lemma 1, every match set is represented exactly as a finite union of boxes. Emptiness of a box is decided exactly per field: an interval is empty iff its bounds cross, or they touch with an open end; for integers, iff it contains no integer (computed by rounding the bounds inward); a boolean set is empty iff it has no element. Intersection is exact fieldwise. The differences $X \setminus \bigcup_j Y_j$ are computed by iterating Lemma 2 over each Y_j and each remaining piece, which is exact by induction on j . Each predicate is either a non-emptiness of an intersection or difference, or a containment $X \subseteq Y \iff X \setminus Y = \emptyset$. All are therefore decided exactly. $\square$

Theorem 2 (Witness soundness). *For every reported conflict or invariant violation, the witness w returned lies in the corresponding non-empty set. The evaluator of Definition 1 on w returns block for a block/allow conflict; the newer modifier's parameters for a modifier conflict; and not block for an invariant violation.*

Proof. The witness is built fieldwise from a non-empty box. For an interval it returns a preferred value clamped into the interval: an interior point, a closed endpoint, or, for an open endpoint, a point strictly inside. For integers, the rounded preference is clamped into the non-empty integer range. For booleans, an element of the set. So w lies in the box, and hence in the set. For a block/allow conflict, $w \in \mathcal{M}(b)$ for a block b , so Definition 1 returns block. For a modifier conflict, $w \notin \mathcal{B}$ and $w \in \mathcal{M}(m')$ with m' newer than m . No block matches, so the newest matching modifier applies; it is m' or an even newer one, but never m . For an invariant, $w \notin \mathcal{B}$, so no block rule of δ matches. $\square$

Proposition 1 (Complexity). *Let a device have m rules with at most k atoms each. Every box that arises during the analysis is a union of cells of a grid with at most $2mk + 3$ cells per field. Hence the number of pairwise disjoint boxes representing any intermediate set is at most $(2mk + 3)^d$, and each finding is decided in time polynomial in m for fixed d and k .*

Proof. All interval endpoints come from atom thresholds or the two domain bounds, so each field has at most $mk + 2$ distinct endpoints. A set of n points on a line cuts it into n singletons and at most $n - 1$ open segments between them, and D_f splits into at most $2(mk + 2) - 1 = 2mk + 3$ elementary pieces. Every box produced by intersection or by Lemma 2 has endpoints among these thresholds, so it is a union of product cells. Disjoint boxes, each containing at least one cell, number at most the cell count, $(2mk + 3)^d$. Each step of Lemma 2 is $O(d)$, which gives the polynomial bound. $\square$

With $d = 6$ the bound is loose. In practice rule sets are small and most rules touch one or two fields (Section V).

TABLE I
MEDIAN ANALYSIS TIME (MS) PER RULE SET, SINGLE DEVICE.

Rules	Interval checker	Z3 encoding	Speed-up
5	0.41	16.1	39×
10	1.68	43.2	26×
20	7.08	125.3	18×
40	22.0	354.1	16×
80	85.1	1181.9	14×

IV. SYSTEM

The checker (`rule_set_checker.py`) has no dependencies and runs inside the authoring Lambda. After the LLM compiler and the per-rule validator, the new rule is analyzed as the newest member of its device’s active policies:

- an *unsatisfiable* rule, or an *invariant* violation (invariants are configured as a list of (δ, A_I) pairs), returns HTTP 422 and the rule is not stored;
- conflicts, shadowing and redundancy are stored and returned under `analysis`, each with its witness context, so the author sees, for example, that an existing block overrides their new allow.

Witnesses prefer typical values (70 °F, noon) when the region permits, so a counterexample reads as a plausible afternoon rather than −80 °F at midnight.

Independent SMT encoding. A separate script encodes the same predicates in Z3: fields are Real/Int/Bool variables with domain constraints, rules are conjunctions, and the evaluator’s priority is spelled out as formulas. It is not in the serving path. It exists so that the interval implementation is checked against a solver rather than against our own reading of it. If the DSL gains a cross-field condition, the box argument stops holding, and this encoding is the one to keep.

V. EVALUATION

Agreement. We generated 1,000 random rule sets of 1–6 rules over two devices. Thresholds were drawn to include boundary cases (hour 24, humidity 101, equal and not-equal on the same value), with a heater-when-away invariant. The findings, compared as (kind, rules) pairs, were *identical* between the interval checker and Z3 on all 1,000 sets. The agreement claim covers sets of 1–6 rules; the larger sets of Table I were timed on both but not compared finding by finding. The out-of-domain thresholds are deliberate: hour 24 and humidity 101 lie outside their fields’ domains, and they check that both implementations intersect every atom with its field’s domain rather than trusting the threshold. The sets produced 544 unsatisfiable rules, 966 invariant violations, 265 block/allow conflicts, 122 modifier conflicts, 101 redundancies and 61 shadowed modifiers.

Witness replay. Every witness for a block/allow conflict (265), a modifier conflict (122) or an invariant violation (966) was replayed through the *production* evaluator, and all 1,353 reproduced the finding. For modifier conflicts this means the verdict was *modify* and the older modifier’s parameters were not the ones applied. Unsatisfiable, shadowed and redundant

findings are claims about the absence of contexts, so they have no witness to replay; they are covered by the Z3 agreement. A mutation that made $>$ inclusive failed both the agreement test and the replay test.

Cost. Table I: the checker analyzes a realistic household (5–20 rules per device) in about 7 ms or less, and 80 rules in under 0.1 s, 14–39× faster than the Z3 encoding. Timings are medians on one x86_64 machine; the Z3 column includes Python encoding overhead. Growth is superlinear, as Proposition 1 allows, but far below the bound.

VI. A BENCHMARK FOR LLM POLICY COMPILATION

The analysis catches errors *between* rules. The remaining question is how often the compiler gets a single rule wrong in a way the validator accepts. We release 380 plain-English rules with gold policies:

- 200 single-condition rules (20 condition phrasings × 5 action phrasings × 2 devices), including explicit numbers and the semantic shorthands the compiler’s prompt defines;
- 120 two-condition conjunctions;
- 20 rules with a *modify* action;
- 20 rules that must be refused (unsupported devices or conditions, vague requests);
- 20 self-contradictions.

Scoring is by *meaning*: a compiled rule is correct iff it has the gold device and action, and $\mathcal{M}(\text{compiled}) = \mathcal{M}(\text{gold})$. That equality is decided by Theorem 1 as two empty differences, so *hour* ≥ 22 and *hour* > 21 score as equal, and > 85 vs. ≥ 85 as different. The headline metric is the *silent error rate*: among compiled rules the validator accepts, the fraction that mean something else.

The validator’s blind spot, measured. Compiled faithfully, all 20 self-contradictions (20/20) pass the per-rule validator. They are well-formed, typed and confident, and only the set-level analysis rejects them.

Measuring accuracy and the silent error rate of specific LLM compilers requires running each model over the benchmark, and we leave it to future work. The released harness calls the deployed compiler directly, or scores saved compiler outputs offline, so results for any model can be reproduced from its outputs alone. The contradiction result above needs no model: it is a property of the validator, and it holds for any compiler that renders these rules faithfully.

VII. LIMITATIONS

The analysis assumes every context field is present. A missing field can only stop a rule matching, so a block may fail to fire, and the evaluator treats that as no match. Invariants are user-supplied; the system checks them but does not infer them. The benchmark’s phrasings are templated from the compiler’s own semantic reference. It therefore measures fidelity to that reference, not robustness to free paraphrase, which a human-written split would test.

VIII. CONCLUSION

Because the policy language is unary per field, whole-rule-set verification is exact, fast and dependency-free. Every finding comes with a witness the production evaluator confirms, and an independent SMT encoding agrees on 1,000 random sets. The check now stands between an LLM’s compilation and a policy going live.

REFERENCES

- [1] T. Raja, “DeviceWeave: Natural-language policy authoring and device control for smart homes,” GitHub repository, 2026. [Online]. Available: <https://github.com/rajatarun/DeviceWeave>
- [2] L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Proc. TACAS*, 2008, pp. 337–340.
- [3] Z. B. Celik, P. McDaniel, and G. Tan, “Soteria: Automated IoT safety and security analysis,” in *Proc. USENIX ATC*, 2018, pp. 147–158.
- [4] D. T. Nguyen, C. Song, Z. Qian, S. V. Krishnamurthy, E. J. M. Colbert, and P. McDaniel, “IoTSan: Fortifying the safety of IoT systems,” in *Proc. CoNEXT*, 2018, pp. 191–203.
- [5] E. S. Al-Shaer and H. H. Hamed, “Discovery of policy anomalies in distributed firewalls,” in *Proc. IEEE INFOCOM*, 2004, pp. 2605–2616.